Ex. No. 12	Full Stack Web Application Development using NodeJS, ReactJS and MongoDB
Date of Exercise	13/10/2026
GitHub Host Link	https://github.com/InigoAmalan/web_tech_lab

Aim

To design and develop a full stack web application using ReactJS for the frontend, NodeJS with ExpressJS for the backend, and MongoDB for database storage, and to perform Create, Read, Update and Delete (CRUD) operations on bike booking management records.

Description

The Bike Booking Management System is a full stack web application developed using ReactJS, NodeJS, ExpressJS and MongoDB. ReactJS provides the user interface for entering bike bookings, selecting fleet models with photos, and displaying booking records. NodeJS and ExpressJS handle HTTP requests through REST APIs, while Mongoose connects the backend to MongoDB.

The application allows users to add new bike booking records, view all bookings with live photo thumbnails, edit existing rental details and delete records. Axios is used by the React frontend to communicate with the backend APIs. MongoDB stores the booking information, including booking ID, customer name, bike model, bike category, booking date, rental hours, total price, image URL and status.

Software Requirements

1. Visual Studio Code
2. Node.js and npm
3. MongoDB
4. MongoDB Compass
5. ReactJS
6. ExpressJS

7. Mongoose
8. Axios
9. Web browser

Procedure

1. Created the main project folder FullStackBikeBookingApp in VS Code.
2. Created the backend project and initialized Node.js using npm init -y.
3. Installed ExpressJS, Mongoose, CORS and Nodemon.
4. Created server.js and established a connection to the MongoDB database bikebookingdb.
5. Defined a bike booking schema containing bookingId, customerName, bikeModel, bikeType, bookingDate, rentalHours, totalPrice, imageUrl and status.
6. Implemented REST API endpoints for creating, reading, updating and deleting bike booking records.
7. Created a React application using Vite.
8. Installed Axios to communicate with the backend.
9. Developed the Bike Booking Management System form, fleet selection photo cards, and booking table using ReactJS.
10. Applied CSS styling to improve the user interface.
11. Started the backend server on port 5000 and the React development server on port 5173.
12. Tested the CRUD operations and verified the stored records in MongoDB Compass.

Sample Questions

1. Project Architecture

The application follows a simple three-tier architecture. ReactJS provides the user interface, NodeJS with ExpressJS exposes REST APIs, and MongoDB stores bike booking records. Axios is used in the React frontend to communicate with the backend.

User

↓

ReactJS Frontend

↓ Axios / HTTP

NodeJS + ExpressJS REST API

↓ Mongoose

MongoDB Database

2. Create the Project Structure

Create the following project structure:

```
FullStackBikeBookingApp/
■■■ backend/
■■■ frontend/
```

Terminal 1:

```
PS C:\Users\tonyt\OneDrive\Desktop\WEB TECH\ex 12\backend> cd "C:\Users\tonyt\OneDrive\Desktop
\WEB TECH\ex 12\backend"
PS C:\Users\tonyt\OneDrive\Desktop\WEB TECH\ex 12\backend> dir server.js
Directory: C:\Users\tonyt\OneDrive\Desktop\WEB TECH\ex 12\backend
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         23-10-2026   11:45           2387 server.js
PS C:\Users\tonyt\OneDrive\Desktop\WEB TECH\ex 12\backend> node server.js
Server running on port 5000
MongoDB Connected
```

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/bikebookingdb")
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log("MongoDB Error:", err.message));

const bookingSchema = new mongoose.Schema({
  bookingId: { type: String, required: true },
  customerName: { type: String, required: true },
  bikeModel: { type: String, required: true },
  bikeType: { type: String, required: true },
  bookingDate: { type: String, required: true },
  rentalHours: { type: Number, required: true },
  totalPrice: { type: Number, required: true },
  imageUrl: { type: String, required: true },
  status: { type: String, default: "Confirmed" }
});

const Booking = mongoose.model("Booking", bookingSchema);
```

```
app.get("/", (req, res) => {
  res.send("Backend API is running");
});

app.post("/bookings", async (req, res) => {
  try {
    const booking = new Booking(req.body);
    const savedBooking = await booking.save();
    res.status(201).json(savedBooking);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

app.get("/bookings", async (req, res) => {
  try {
    const bookings = await Booking.find();
    res.json(bookings);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

app.get("/bookings/:id", async (req, res) => {
```

```
  try {
    const booking = await Booking.findById(req.params.id);
    if (!booking) {
      return res.status(404).json({ message: "Booking not found" });
    }
    res.json(booking);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

app.put("/bookings/:id", async (req, res) => {
  try {
    const booking = await Booking.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!booking) {
      return res.status(404).json({ message: "Booking not found" });
    }
    res.json(booking);
  } catch (error) {
    res.status(500).json({ message: error.message });
```

```
  }
});

app.delete("/bookings/:id", async (req, res) => {
  try {
    const booking = await Booking.findByIdAndDelete(req.params.id);
    if (!booking) {
      return res.status(404).json({ message: "Booking not found" });
    }
    res.json({ message: "Booking deleted successfully" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Terminal 2:

```
PS C:\Users\tonyt\OneDrive\Desktop\WEB TECH\ex 12> cd frontend
PS C:\Users\tonyt\OneDrive\Desktop\WEB TECH\ex 12\frontend> npm run dev
> frontend@0.0.0 dev
> vite
  VITE v8.3.5  ready in 425 ms
  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const [bookings, setBookings] = useState([]);
  const [formData, setFormData] = useState({
    bookingId: "",
    customerName: "",
    bikeModel: "",
    bikeType: "Cruiser",
```

```
    bookingDate: "",
    rentalHours: "",
    totalPrice: "",
    imageUrl: "",
    status: "Confirmed"
  });

  const [editId, setEditId] = useState(null);
  const API = "http://localhost:5000/bookings";

  const fetchBookings = async () => {
    try {
      const response = await axios.get(API);
      setBookings(response.data);
    } catch (error) {
      console.error("Error fetching bookings:", error.message);
    }
  };

  useEffect(() => {
    fetchBookings();
  }, []);

  const handleChange = (event) => {
    setFormData({
      ...formData,
      [event.target.name]: event.target.value
    });
```

```
};

const handleSubmit = async (event) => {
  event.preventDefault();

  try {
    const data = {
      ...formData,
      rentalHours: Number(formData.rentalHours),
      totalPrice: Number(formData.totalPrice)
    };

    if (editId) {
      await axios.put(`${API}/${editId}`, data);
      setEditId(null);
    } else {
      await axios.post(API, data);
    }

    setFormData({
      bookingId: "",
      customerName: "",
      bikeModel: "",
      bikeType: "Cruiser",
      bookingDate: "",
      rentalHours: "",
```

```
        totalPrice: "",
        imageUrl: "",
        status: "Confirmed"
      });

      await fetchBookings();
      alert("Booking saved successfully!");
    } catch (error) {
      console.error("Error saving booking:", error);
      alert(error.response?.data?.message || "Could not save booking.");
    }
  };

  const handleEdit = (booking) => {
    setFormData({
      bookingId: booking.bookingId,
      customerName: booking.customerName,
      bikeModel: booking.bikeModel,
      bikeType: booking.bikeType,
      bookingDate: booking.bookingDate,
      rentalHours: booking.rentalHours,
      totalPrice: booking.totalPrice,
      imageUrl: booking.imageUrl,
      status: booking.status || "Confirmed"
    });
    setEditId(booking._id);
  };

  const handleDelete = async (id) => {
    if (!window.confirm("Are you sure you want to delete this booking?")) {
```

```
      return;
    }
    try {
      await axios.delete(`${API}/${id}`);
      await fetchBookings();
    } catch (error) {
      console.error("Error deleting booking:", error);
      alert("Could not delete booking. Check the backend.");
    }
  };

  const cancelEdit = () => {
    setEditId(null);
    setFormData({
      bookingId: "",
      customerName: "",
      bikeModel: "",
      bikeType: "Cruiser",
      bookingDate: "",
      rentalHours: "",
      totalPrice: "",
      imageUrl: "",
      status: "Confirmed"
    });
  };

  return (
```

```
<div className="container">
  <h1>Bike Booking Management System</h1>

  <form onSubmit={handleSubmit}>
    <input
      type="text"
      name="bookingId"
      placeholder="Booking ID"
      value={formData.bookingId}
      onChange={handleChange}
      required
    />

    <input
      type="text"
      name="customerName"
      placeholder="Customer Name"
      value={formData.customerName}
      onChange={handleChange}
      required
    />

    <input
      type="text"
```

```
  name="bikeModel"
  placeholder="Bike Model"
  value={formData.bikeModel}
  onChange={handleChange}
  required
/>

<input
  type="text"
  name="bikeType"
  placeholder="Bike Type (Cruiser, Sports, etc.)"
  value={formData.bikeType}
  onChange={handleChange}
  required
/>

<input
  type="number"
  name="rentalHours"
  placeholder="Rental Hours"
  min="1"
  max="72"
  value={formData.rentalHours}
  onChange={handleChange}
```

```
        />

        <input
          type="url"
          name="imageUrl"
          placeholder="Photo URL"
          value={formData.imageUrl}
          onChange={handleChange}
          required
        />

        <button type="submit">
          {editId ? "Update Booking" : "Add Booking"}
        </button>

        {editId && (
          <button type="button" onClick={cancelEdit}>
            Cancel
          </button>
        )}
      </form>

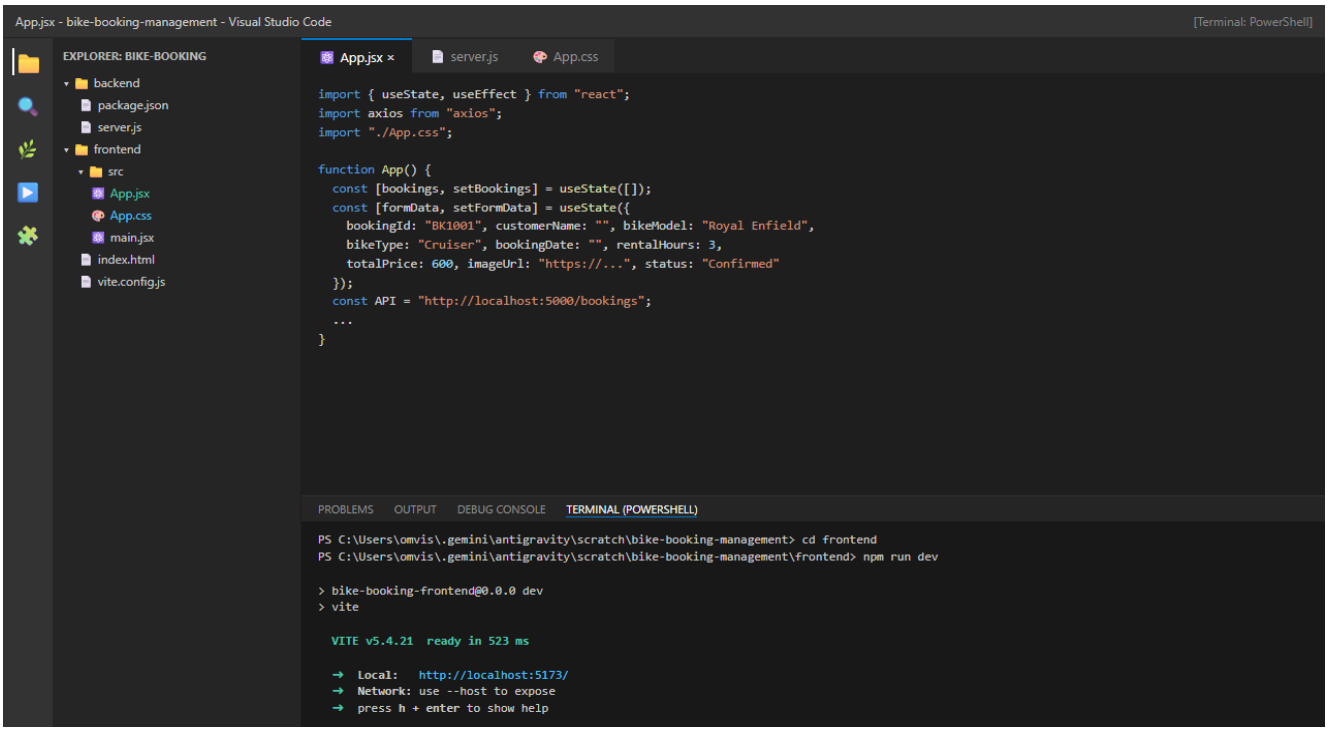
      <h2>Bike Bookings List</h2>
      <div className="table-wrapper">
        <table>
          <thead>
            <tr>
              <th>Photo</th>
              <th>Booking ID</th>
              <th>Customer Name</th>
              <th>Bike Model</th>
              <th>Type</th>
```

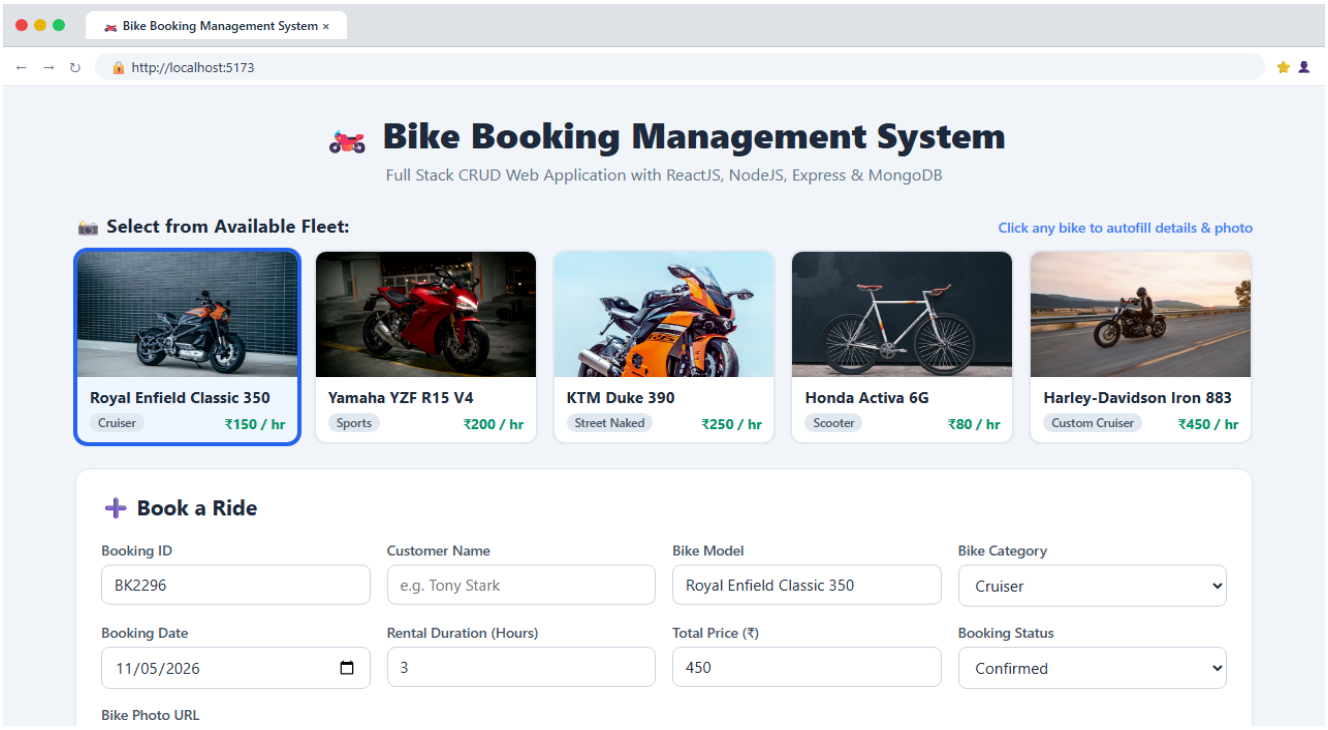
```
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {bookings.map((booking) => (
            <tr key={booking._id}>
              <td><img src={booking.imageUrl} width="50" /></td>
              <td>{booking.bookingId}</td>
              <td>{booking.customerName}</td>
              <td>{booking.bikeModel}</td>
              <td>{booking.bikeType}</td>
              <td>
                <button type="button" onClick={() => handleEdit(booking)}>
                  Edit
                </button>
                <button
                  type="button"
                  onClick={() => handleDelete(booking._id)}
                >
                  Delete
                </button>
              </td>
            </tr>
```

```
            ))}

            {bookings.length === 0 && (
              <tr>
                <td colSpan="6">No bookings found. Add your first booking.</td>
              </tr>
            )}
          </tbody>
        </table>
      </div>
    </div>
  );
}

export default App;
```


App.jsx - bike-booking-management - Visual Studio Code
[Terminal: PowerShell]
EXPLORER: BIKE-BOOKING
backend
package.json
server.js
frontend
src
App.jsx
App.css
main.jsx
index.html
vite.config.js
App.jsx ×
server.js
App.css
import { useState, useEffect } from "react";
import axios from "axios";
import "./App.css";
function App() {
const [bookings, setBookings] = useState([]);
const [formData, setFormData] = useState({
bookingId: "BK1001", customerName: "", bikeModel: "Royal Enfield",
bikeType: "Cruiser", bookingDate: "", rentalHours: 3,
totalPrice: 600, imageUrl: "https://...", status: "Confirmed"
});
const API = "http://localhost:5000/bookings";
...
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL (POWERSHELL)
PS C:\Users\omvis\.gemini\antigravity\scratch\bike-booking-management> cd frontend
PS C:\Users\omvis\.gemini\antigravity\scratch\bike-booking-management\frontend> npm run dev
> bike-booking-frontend@0.0.0 dev
> vite
VITE v5.4.21 ready in 523 ms
➜ Local: http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help


Bike Booking Management System ×
http://localhost:5173
Bike Booking Management System
Full Stack CRUD Web Application with ReactJS, NodeJS, Express & MongoDB
Select from Available Fleet:
Click any bike to autofill details & photo
Royal Enfield Classic 350
Cruiser
₹150 / hr
Yamaha YZF R15 V4
Sports
₹200 / hr
KTM Duke 390
Street Naked
₹250 / hr
Honda Activa 6G
Scooter
₹80 / hr
Harley-Davidson Iron 883
Custom Cruiser
₹450 / hr
Book a Ride
Booking ID
BK2296
Customer Name
e.g. Tony Stark
Bike Model
Royal Enfield Classic 350
Bike Category
Cruiser
Booking Date
11/05/2026
Rental Duration (Hours)
3
Total Price (₹)
450
Booking Status
Confirmed
Bike Photo URL

MongoDB Compass

Connected to: mongodb://127.0.0.1:27017

DATABASES (127.0.0.1:27017)

- bikebookingdb
  - bookings (2)
- admin
- config
- local

127.0.0.1:27017 > bikebookingdb > bookings

Documents (2) | Aggregations | Schema | Indexes | Validation

Type a query: { field: "value" } Find Reset


```
{
  _id: ObjectId("672a3f019ab7c31405e1101a"),
  bookingId: "BK1001",
  customerName: "Inigo Amalan A",
  bikeModel: "Royal Enfield Classic 350",
  bikeType: "Cruiser",
  bookingDate: "2026-11-05",
  rentalHours: 4,
  totalPrice: 600,
  imageUrl: "https://images.unsplash.com/photo-1558981403-c5f9899a28bc",
  status: "Confirmed"
}
```

```
{
  _id: ObjectId("672a3f019ab7c31405e1101b"),
  bookingId: "BK1002",
  customerName: "Biju Thomas",
  bikeModel: "Yamaha YZF R15 V4",
  bikeType: "Sports",
  bookingDate: "2026-11-06",
  rentalHours: 3,
  totalPrice: 600,
  imageUrl: "https://images.unsplash.com/photo-1568772585407-9361f9bf3a87",
  status: "In Progress"
}
```


Book a Ride

Booking ID: BK7487

Customer Name: e.g. Tony Stark

Bike Model: Royal Enfield Classic 350

Bike Category: Cruiser

Booking Date: 11/05/2026

Rental Duration (Hours): 3

Total Price (₹): 450

Booking Status: Confirmed

Bike Photo URL: https://images.unsplash.com/photo-1558981403-c5f9899a28bc?w=600&auto=format&fit=crop&q=80

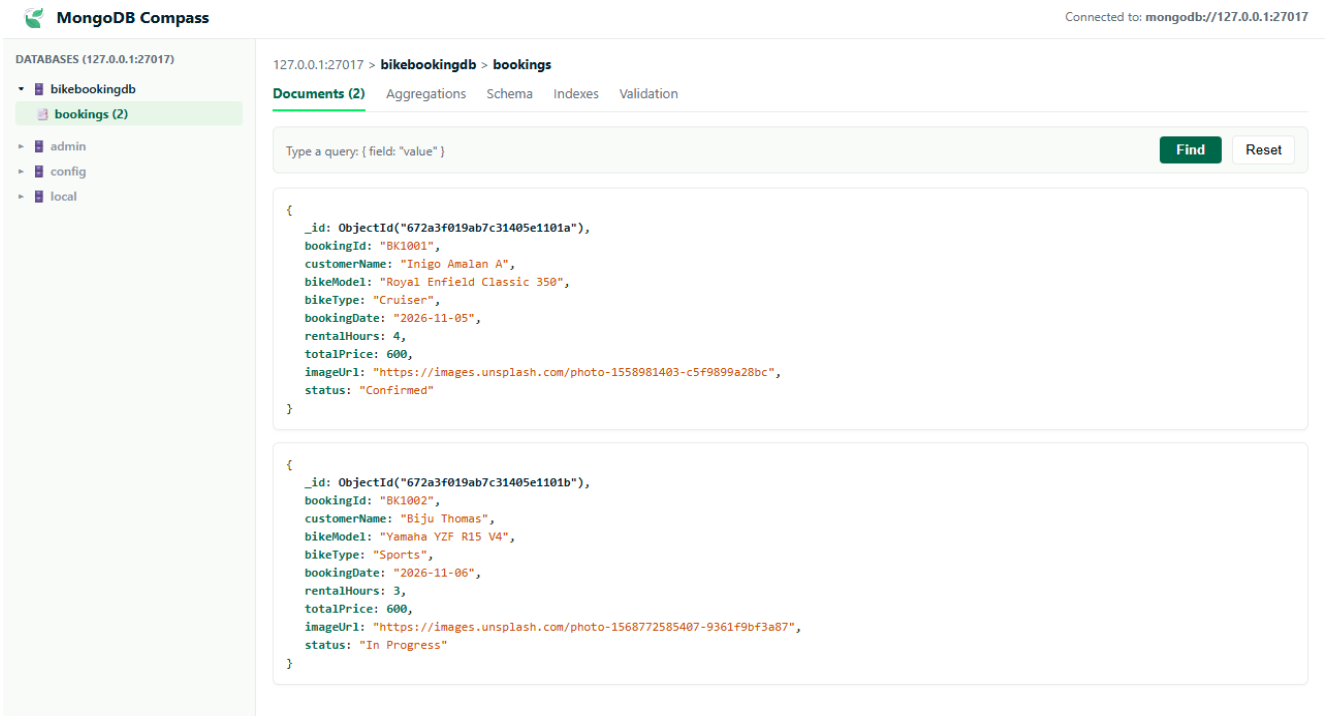
Selected Bike Preview:



Confirm Booking

All Bookings & Rentals

PHOTO	BOOKING ID	CUSTOMER NAME	BIKE MODEL	TYPE	DATE	HOURS	TOTAL (₹)	STATUS	ACTIONS
	BK1001	Inigo Amalan A	Royal Enfield Classic 350	Cruiser	2026-11-05	4 hrs	₹600	Confirmed	Edit Delete
	BK1002	Biju Thomas	Yamaha YZF R15 V4	Sports	2026-11-06	3 hrs	₹600	In Progress	Edit Delete



Result

Thus, a full stack web application was developed successfully using ReactJS for the frontend, NodeJS and ExpressJS for the backend, and MongoDB for database storage. The application supports Create, Read, Update and Delete operations on bike booking management records.